storygit

Version control for story state

Arush Gumber

Abstract

storygit is an AI-assisted storytelling system that develops a one-line premise into a structured serial — Story, Episodes, Scenes, Beats, Prose — while keeping the writer in authority over every change. Its claim is that the hard part of long-form AI-assisted fiction is not generation but *state*: what is true, who knows it, what depends on it, and what a given edit therefore invalidates. storygit models the story as a versioned typed state graph, makes every AI action a reviewable diff against it, propagates edits by *marking* affected nodes rather than rewriting them, checks continuity graph-first before it ever asks a model, and learns the writer's taste from their accept, reject, and edit signals. This document states the problem, the design, the decisions and their rejected alternatives, and the measurements.

1 The problem

A writer starts with a sentence: *a powerless orphan discovers an ability that could change the balance of power in a world at war.* They want to end with a serial — tens or hundreds of episodes, coherent, in their voice, with the mechanics that make a listener press *next episode*. The naive form of AI assistance, prompt to story, fails at this in five reliable ways.

State drift. Facts established early are forgotten or contradicted later. The model has no ledger of what it has committed to, so episode 14 puts a character in a city they left in episode 9. The sharpest version of this failure is *epistemic*: a character acts on information the story has never given them. A reader forgives an inconsistent hair colour; they do not forgive a character who somehow already knows the secret.

Broken edit semantics. The writer changes one scene. Either nothing downstream responds — the story now contains a contradiction nobody flagged — or the system regenerates the plan and destroys work the writer had already approved. Both come from the same absence: nothing records what depends on what. Fabula's own study [3] reports exactly this, from several writers independently.

Agency erosion. When the system proposes whole drafts, the writer's role collapses into judging them. Fabula's participants described becoming "an arbiter of good or bad" rather than an author [3]. That is a worse job than writing, and it is the reason a tool can be impressive in a demo and unused in a month.

Exploration overwhelm. Offering more alternatives is not the same as offering more choice. Unlabeled variants cost more to read than they save, and a writer will stop opening them.

No learning. The thirtieth iteration repeats the first iteration's mistakes. Nothing in the system is a model of *this* writer's taste, so every correction has to be made again.

To these, serialized audio fiction adds a sixth: **no serial mechanics**. Hooks, cliffhangers, recaps, and the open threads a listener is waiting to see paid off are the retention surface of the product, and a general-purpose story generator does not track any of them.

2 Thesis

Treat the story as a **versioned, typed state graph**. Make every AI action a **reviewable diff** against that graph rather than a block of replacement text. Make dependencies **explicit**, so that an edit propagates by *marking* what it affected and never by rewriting it. Let the writer **own the objective**, in their own words, rather than ranking against the system's. And **learn the writer's taste** from the signals they are already producing — what they accept, what they reject, and how they edit.

The division of labour follows: **the AI handles coherence and mechanics; the human handles taste, voice, and retention.**

Stated as a difference: Fabula is a chain of prompts with a user interface. storygit is a state machine with a learning loop. The chain is fast to build and pleasant to demonstrate; the state machine is what survives episode 40.

3 Fabula, and the gaps this exploits

Fabula [3] is the closest published system, and the immediate predecessor of this design's questions; Dramatron [2] and Wordcraft [4] are the hierarchical-generation and writer-in-the-loop lines it builds on, and the creativity-support literature [1] supplies the evaluation vocabulary.

[Written in chunk 2 — summary of the Fabula system, its user study findings, and the five gaps storygit targets: no dependency tracking, no per-character knowledge state, no learning from writer signals, no serial mechanics, and diversity implemented as "do not repeat the last suggestion".]

4 Architecture

[Full system diagram and component walk-through — chunk 6. Model routing subsection — chunk 2.]

5 The state model

Five things are stored, and everything else is derived from them.

The plan tree. Story, Episode, Scene, Beat, Prose. Every node carries the four planning questions Fabula found writers actually use — what happens, what the audience learns, how they feel, where and when — plus a review status (draft, accepted, locked, stale) and, when stale, the reason. Episode nodes add the serial mechanics: hook, cliffhanger, recap of the previous episode, threads carried in and left out, writer-set goals, and a target length. Beat nodes add produces and consumes — the declared fact dependencies that make propagation exact.

The world graph. Entities (characters, places, objects, factions) with an alias table, and facts as edges between them. A fact carries a validity interval over the global beat order, so a fact that changes does not overwrite its own history: Kael is in Ashfall for beats 3 through 7 and elsewhere afterwards, and both are retrievable at their own times. Predicates come from a closed vocabulary (location, alive, injury, possesses, relationship, relationship_status, goal, secret, trait, faction) plus a free-text note. Closed is what makes the first layer of continuity checking a dictionary lookup rather than a model call.

Epistemic edges. knows(character, fact, since beat), tracked separately from the facts themselves, because the difference between what is true and what a character has been told is the single richest source of continuity errors in generated fiction.

The open-thread ledger. Each thread records when it was opened and last advanced, so "you opened the sister subplot in episode 2 and have not touched it in nine episodes" is a query, not a judgement.

The writer ledger. Locks, hard constraints, style notes, writer-defined scoring criteria in the writer's own words, rejected directions, and the coherent-to-surprising dial. This is the object that keeps the human in authority, and everything in it is visible and deletable — including the rules the system mined from the writer's own edits, which must never influence generation invisibly.

Provenance is recorded per sentence (`human`, `ai`, `ai_edited_by_human`), because the interesting case is mixed: the AI proposed a paragraph and the writer rewrote two sentences of it. A node-level flag would lose exactly the information the writer cares about, and the authorship ratio shown in the interface would be a lie.

5.1 Snapshots, branches, and why the project is called storygit

Every object is stored once under the SHA-256 of its canonical JSON. A snapshot is a *manifest*: a mapping from logical id to content hash, plus a parent pointer, the author, and the intent line. A branch is a name pointing at a snapshot.

Three properties fall out for free. Versions are cheap: committing a change that touches three nodes writes three object rows and one manifest, so a five-hundred-snapshot history of a two-hundred-node story costs the changed nodes, not five hundred copies. Structural diff needs no heuristics: what changed between two versions is a set comparison over two manifests. And the whole thing is reproducible: identical content yields an identical manifest, so "apply the same diff twice and get the same hashes" is a test rather than a hope.

Merging is three-way at object granularity. Where only one side changed an object, that side wins. Where both changed it differently, the merge returns a conflict. It never guesses which version of a scene the author meant.

6 The proposal engine

[Chunk 2: proposals as diffs, progressive levels, structured output. Chunk 3: axis-conditioned sampling, MMR and DPP selection, the dial.]

7 Propagation

Dependency edges are declared, not inferred: a beat states the facts it establishes and the facts it relies on, and an edge from beat A to beat B exists exactly when B consumes something A produced. When an accepted change alters or removes a fact, the system walks the consumers transitively and *marks* them. It never rewrites them.

Three rules make this safe to leave running.

1. **Locked nodes are never marked**, and the walk does not pass through them. A locked beat is not going to change, so nothing it produces can change either. The operational meaning of a lock is broader than that: every fact a locked node establishes is exported into every subsequent prompt as a hard constraint.
2. **Human prose is flagged for review, not staled.** The system does not tell an author that their own sentences are out of date; it tells them something they relied on moved, and leaves the judgement where it belongs.

3. **Nothing is ever rewritten.** The writer chooses: regenerate (previewed as a diff, respecting locks and current facts), edit by hand, or dismiss — “it still works”. Regenerating a whole stale subtree exists, as an explicit, confirmed, previewed action.

Because the marks are emitted as ordinary status operations, staleness flows through the same snapshot machinery as every other change. There is no side channel, and the history shows why each node was marked and by which upstream fact.

The same walk, run against a candidate that has not been accepted, is what produces the line the writer reads under every proposal: *“would mark 2 downstream beats stale”*. The blast radius is visible before the decision, not after it.

8 Continuity checking

[Chunk 3: three layers — deterministic graph checks, local NLI for free-text facts, and an LLM soft judge — with the recall each layer contributes and why the order is deterministic-first.]

9 The preference layer

[Chunk 4: exemplar retrieval, edit-pair mining, the contrastive voice model, edit direction vectors, the Bradley-Terry preference head, and Thompson sampling over safe and exploratory slots. Includes the honest paragraph about the data regime.]

10 Evaluation

[Chunk 5: simulated writers with hidden weight vectors, injected ground-truth contradictions, the metric definitions, the ablation table, and the curves.]

11 Decisions, and what was rejected

11.1 Diffs rather than replacement text

Every AI action is a typed operation list against the current state, not a block of prose that overwrites what was there. *Rejected:* Text-level replacement with a visual diff. It looks similar in the interface and is far weaker underneath: a text diff cannot say “this introduces a character”, cannot be checked against the world graph before it is applied, and cannot tell the writer what it would invalidate.

11.2 Declared dependencies rather than inferred ones

A beat states what it produces and consumes; propagation walks those edges only. *Rejected:* Inferring dependencies with a model. A dependency oracle that is wrong ten percent of the time makes every propagation result untrustworthy, which is worse than no propagation at all — the writer stops reading the marks. Embedding-similarity edges are implemented, but as a clearly weaker “maybe affected” signal and as an ablation to measure, never as the primary mechanism.

11.3 A closed predicate vocabulary

Ten predicates plus a free-text escape hatch. *Rejected:* Open-vocabulary relation extraction. Closed predicates make the first checker layer a dictionary lookup and an equality test — free, instant, and never wrong for the wrong reason. The escape hatch is exactly the set of facts that needs the more expensive NLI check, which is a useful way to have partitioned the problem.

11.4 Content-addressed snapshots

Rejected: Storing a full copy of the story per version, or an event log without materialized states. Full copies do not survive a few hundred snapshots. A pure event log makes "what is true right now" a replay, and makes branch comparison hard. Content addressing gives cheap versions, free structural sharing, and diff as set arithmetic.

11.5 Marking rather than auto-repair

Rejected: Automatically regenerating stale downstream nodes. This is the single most tempting feature in the system and the one that would destroy it. It is precisely what Fabula's writers complained about, and the reason is not technical: a system that rewrites approved work without asking cannot be trusted with a manuscript.

11.6 SQLite

Rejected: Postgres. One writer, one story, a few megabytes, and a file the author can copy is the actual workload. Postgres buys concurrency this system does not have. What would change at PocketFM scale is discussed in the systems documentation.

[Further decisions — provider routing, MMR versus DPP, the preference head's feature set, the simulated-writer evaluation — are added as their chunks land.]

12 Limitations

[Chunk 5: what the simulated-writer evaluation can and cannot measure; the data regime for the preference layer; single-writer assumption; cost model.]

References

[1] Erin Cherry and Celine Latulipe. Quantifying the creativity support of digital tools through the creativity support index. *ACM Transactions on Computer-Human Interaction*, 21(4):21:1–21:25, 2014.

[2] Piotr Mirowski, Kory W. Mathewson, Jaylen Pittman, and Richard Evans. Co-writing screenplays and theatre scripts with language models: Evaluation by industry professionals. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems (CHI '23)*, 2023.

[3] Piotr Mirowski, Ben Wedin, Reinald Kim Amplayo, Rich Galt, Duncan Williams, Rida Qadri, Jaume Sanchez-Elias, Erin Drake-Kajioka, Sian Gooding, Lucia Lopez-Rivilla, Joao G. M. Araujo, Lion Schulz, Satinder Baveja, Shakir Mohamed, Edward Grefenstette, Laura Rimell, and Richard Evans. Fabula: Building a narrative storytelling sidekick with the writers' community, 2026.

[4] Ann Yuan, Andy Coenen, Emily Reif, and Daphne Ippolito. Wordcraft: Story writing with large language models. In *Proceedings of the 27th International Conference on Intelligent User Interfaces (IUI '22)*, 2022.

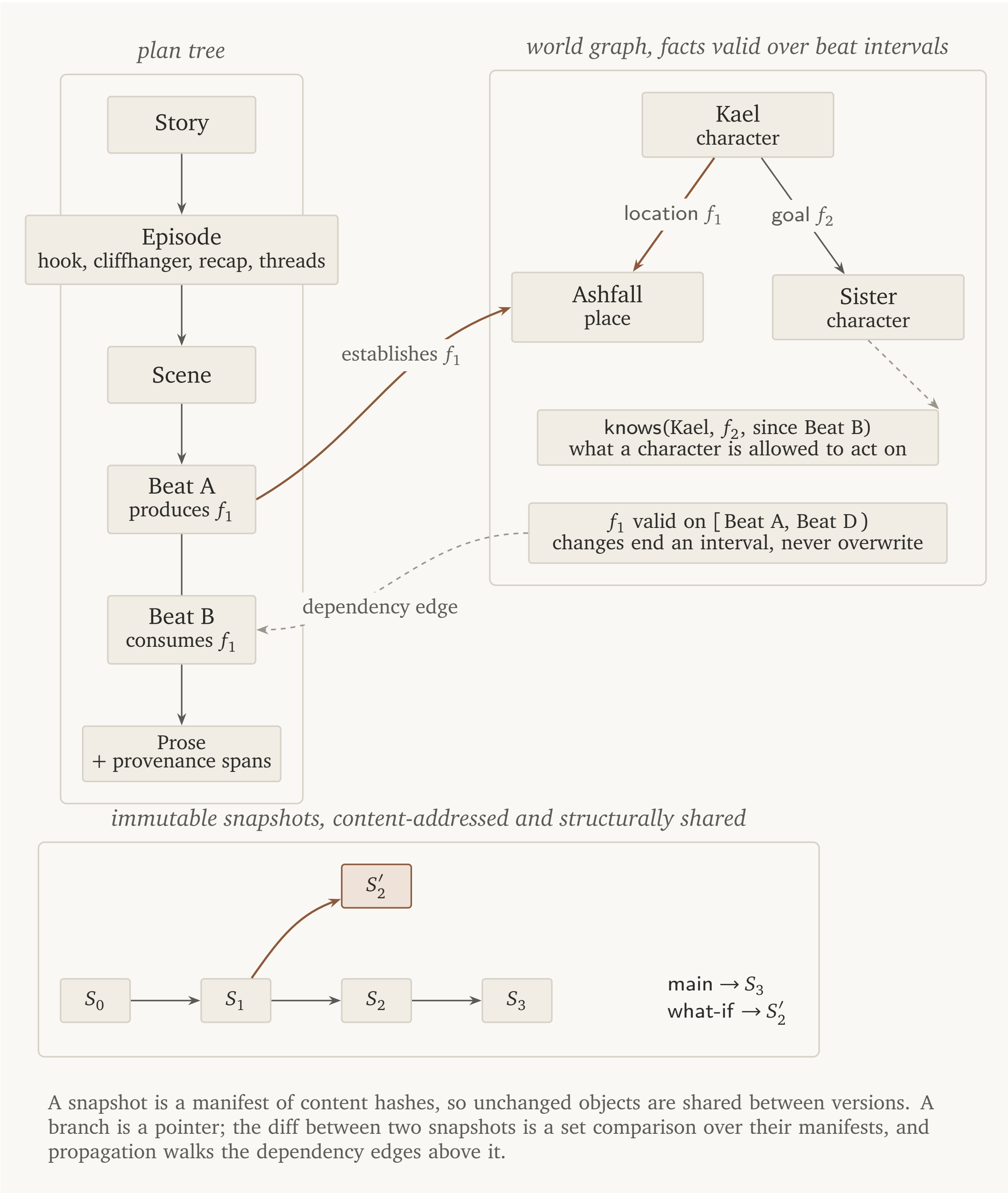

Figure 1: The three stores and how they reference each other. The plan tree holds structure; the world graph holds truth with validity intervals; snapshots hold history. A beat's produces and consumes lists are the only dependency edges propagation walks, which is what makes the walk exact.